Stereo Vision

Fedya Semenov

Advisors: Bill Breslin and Ian Besse

Pacific University

October 26, 2026

Outline

1. Motivation
2. Stereo Geometry & Disparity
3. Rectification & Matching
4. Advanced & Deep Learning Approaches
5. References

Seeing Things Differently

- Depth perception: important

Seeing Things Differently

- Depth perception: important
- Needed by robots, drones, self-driving cars

Seeing Things Differently

- Depth perception: important
- Needed by robots, drones, self-driving cars
- LiDAR:
  - Good, but expensive and bulky
  - Can miss finer details and small objects
  - Can malfunction in fog or rain

Seeing Things Differently

- Depth perception: important
- Needed by robots, drones, self-driving cars
- LiDAR:
  - Good, but expensive and bulky
  - Can miss finer details and small objects
  - Can malfunction in fog or rain
- Humans: good at perceiving depth

Seeing Things Differently

- Depth perception: important
- Needed by robots, drones, self-driving cars
- LiDAR:
  - Good, but expensive and bulky
  - Can miss finer details and small objects
  - Can malfunction in fog or rain
- Humans: good at perceiving depth
- Why can't a machine perceive depth the same way a human does?

Stereo Vision

- Why two eyes?

Stereo Vision

- Why two eyes?
- One eye closed $\rightarrow$ harder to perceive depth

Stereo Vision

- Why two eyes?
- One eye closed $\rightarrow$ harder to perceive depth
- Two eyes $\rightarrow$ easy depth perception, math behind it

Stereo Vision

- Why two eyes?
- One eye closed $\rightarrow$ harder to perceive depth
- Two eyes $\rightarrow$ easy depth perception, math behind it
- For machines: two images from identical cameras placed a short distance apart to perceive depth [3, 4]

Stereo Vision Geometry

- Two cameras separated by baseline B with focal length f
- 3D point P projected onto left and right image planes

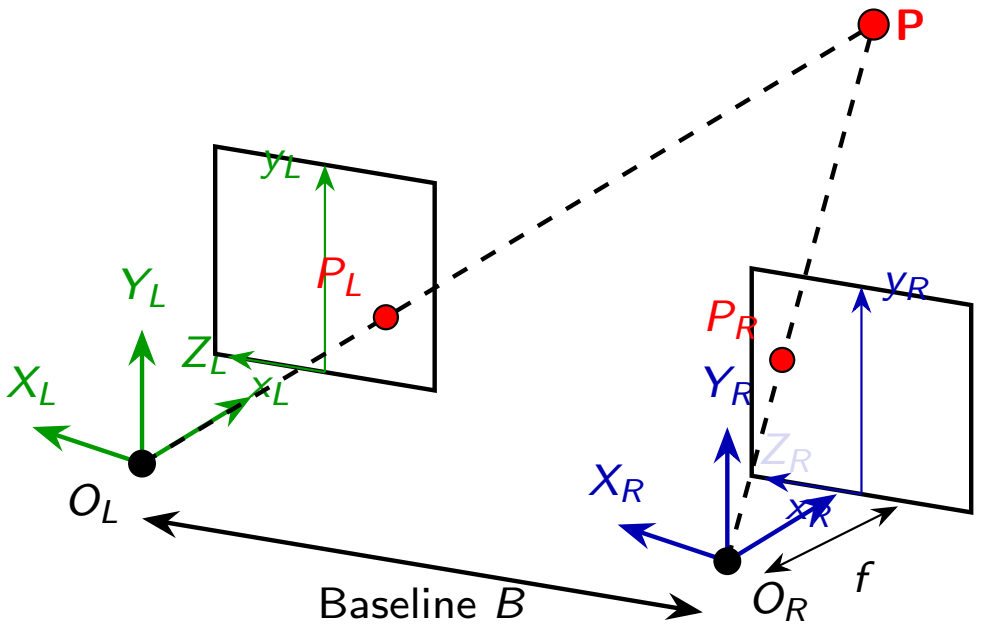


Diagram generated by Gemini [1]

Epipolar Plane & Epipolar Lines

- Epipolar plane: goes through both camera centers (O_L, O_R) and point P in space
- Image projections: P_L and P_R on each camera plane
- Epipolar line: intersection of epipolar plane with image plane

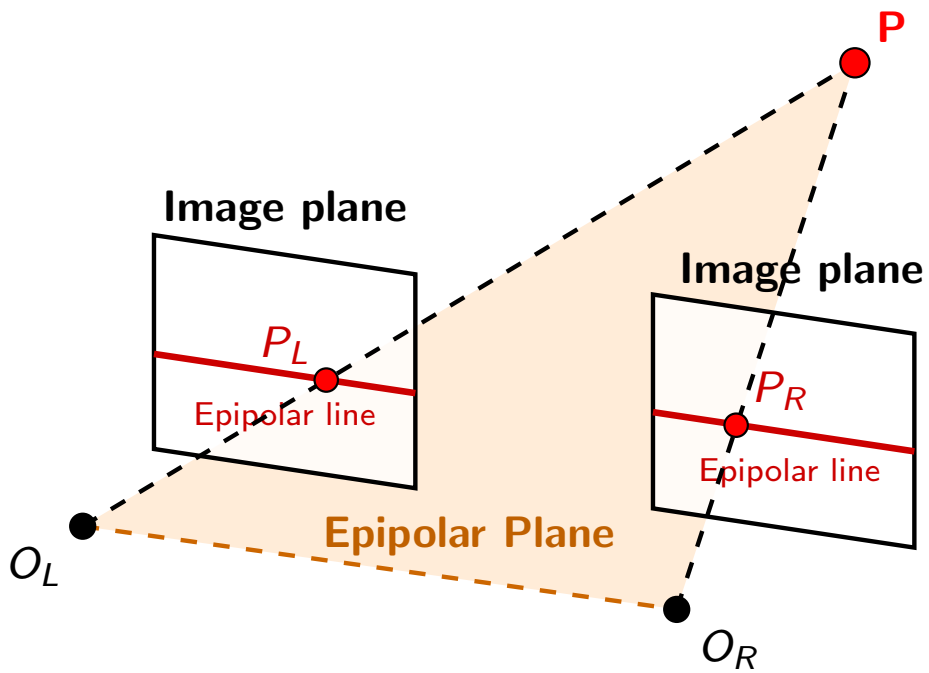


Image Shift & Disparity

- Same 3D point shifted between the two images
- Any point in left image $\rightarrow$ shifted left in right image
- Finger test: hold finger up and close one eye at a time

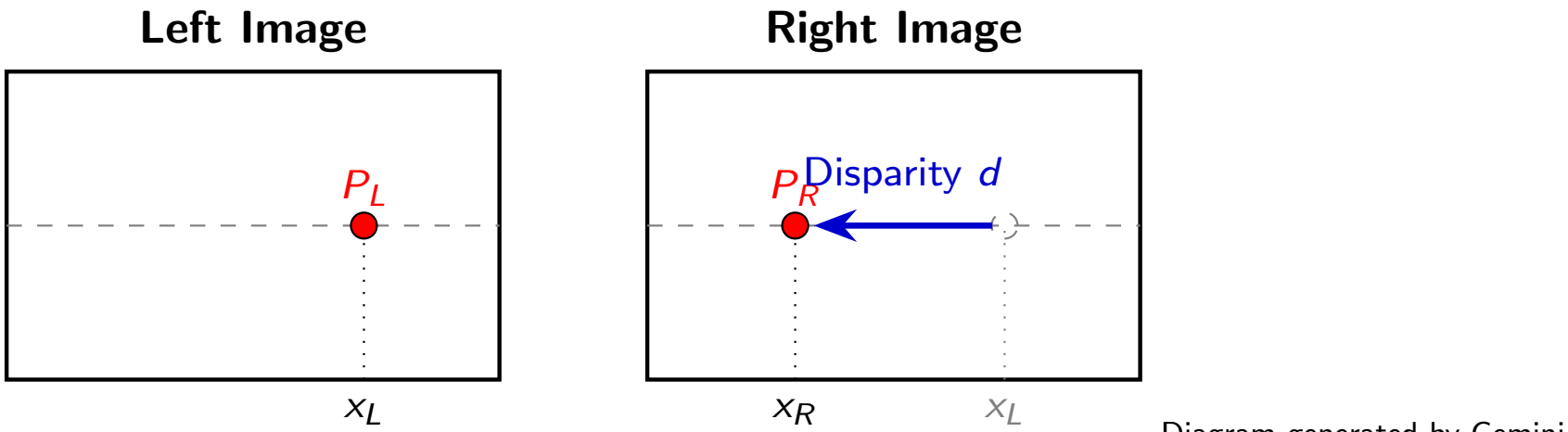


Diagram generated by Gemini

[1]

The "Disparity Trick"

- Disparity (d): shift between left and right image ($d = x_L - x_R$)

The "Disparity Trick"

- Disparity (d): shift between left and right image ($d = x_L - x_R$)
- Points further out $\rightarrow$ smaller disparity

The "Disparity Trick"

- Disparity (d): shift between left and right image ($d = x_L - x_R$)
- Points further out $\rightarrow$ smaller disparity
- Verification: up-close finger shifts dramatically vs. distant object when closing/opening left and right eyes

The "Disparity Trick"

- Disparity (d): shift between left and right image ($d = x_L - x_R$)
- Points further out $\rightarrow$ smaller disparity
- Verification: up-close finger shifts dramatically vs. distant object when closing/opening left and right eyes

Depth Formula

$$Z = \frac{B \cdot f}{d}$$

The "Disparity Trick": Geometry

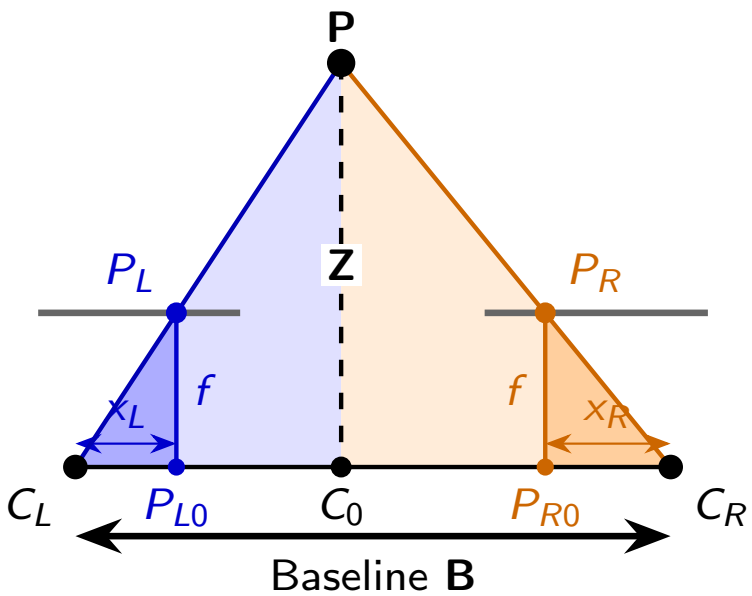


Diagram generated by Gemini [1]

Z: Distance to P
B: Baseline distance
f: Focal length
d: Disparity $(x_L + x_R)$

$$\frac{C_L \to C_0}{Z} = \frac{x_L}{f}$$

$$\frac{C_R \to C_0}{Z} = \frac{x_R}{f}$$

$$\frac{(C_L \to C_0) + (C_R \to C_0)}{Z} = \frac{x_L + x_R}{f}$$

$$\frac{B}{Z} = \frac{d}{f}$$

$$d = \frac{B \cdot f}{Z}$$

$$\mathbf{Z} = \frac{\mathbf{B} \cdot \mathbf{f}}{\mathbf{d}}$$

Rectification

- Want horizontal epipolar lines on both images
- Needed due to:
  - Camera tilt / misalignment
  - Lenses distorting light
- Goal: search along one horizontal line [3]

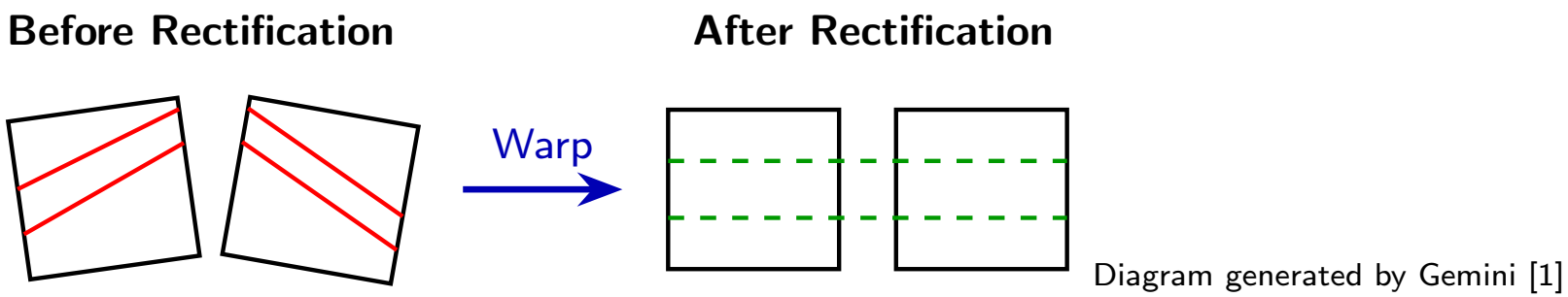


Diagram generated by Gemini [1]

Finding Disparity

- Point in left image + where that point is in right image (disparity) $\rightarrow$ known depth

Finding Disparity

- Point in left image + where that point is in right image (disparity) $\rightarrow$ known depth
- Core problem:
  - Finding best disparity for each pixel [3]
  - Creating a dense depth map from a pair of stereo images [3, 4]

Simplest Algorithm

- Block-based search [3]

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it
- Start at same pixel in right image

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it
- Start at same pixel in right image
- Compare blocks $\rightarrow$ aggregated matching cost

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it
- Start at same pixel in right image
- Compare blocks $\rightarrow$ aggregated matching cost
- Move window along scanline and repeat

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it
- Start at same pixel in right image
- Compare blocks $\rightarrow$ aggregated matching cost
- Move window along scanline and repeat
- Choose pixel with minimum aggregate matching cost as disparity value

Simplest Algorithm

- Block-based search [3]
- For each pixel in left image $\rightarrow$ get block (window) of pixels around it
- Start at same pixel in right image
- Compare blocks $\rightarrow$ aggregated matching cost
- Move window along scanline and repeat
- Choose pixel with minimum aggregate matching cost as disparity value
- Repeat on all pixels

Simplest Algorithm: Block Shifting

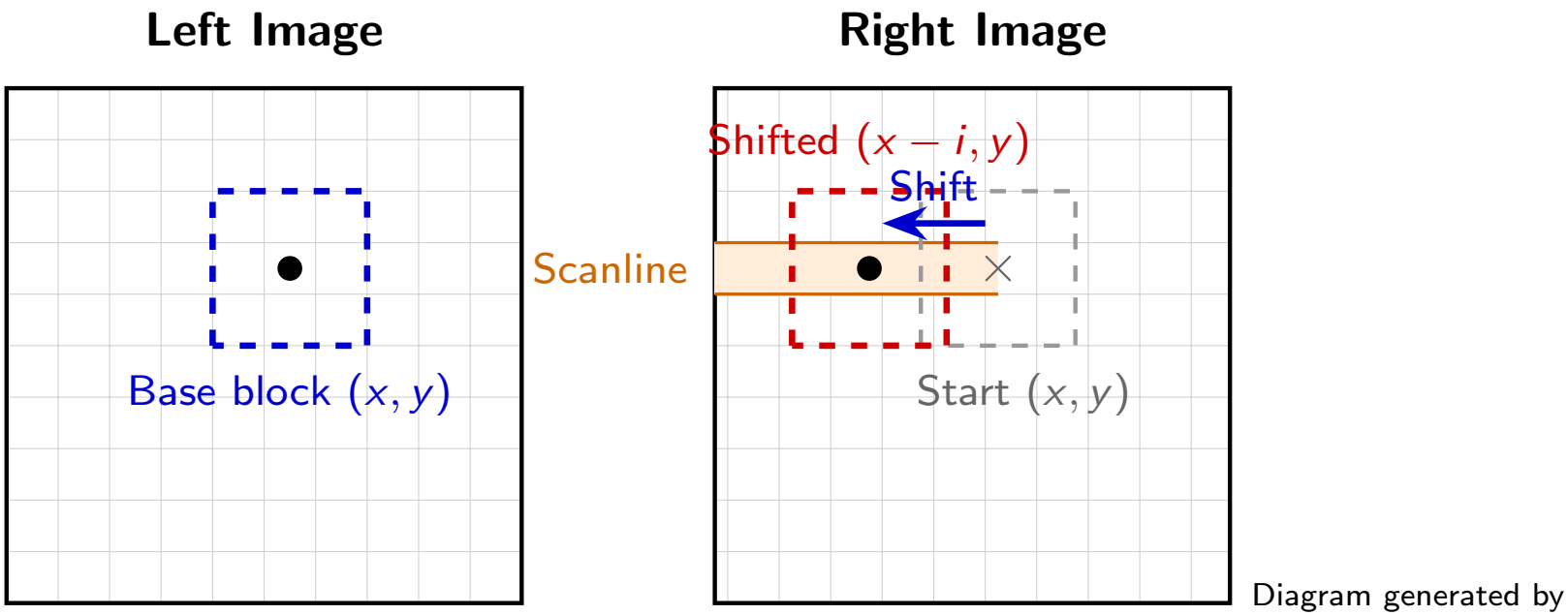


Diagram generated by Gemini [1]

Simplest Algorithm in Python (1/2)

```python
def construct_square(img, x, y, sqsize):
    square = img[y-sqsize:y+sqsize,x-sqsize:x+sqsize]
    return square

def compare_squares(l_square, r_square):
    diff = l_square - r_square
    matching_cost = np.sum(np.abs(diff))
    return matching_cost

def find_best_disparity(l_img, r_img, x, y, sqsize):
    base_l_square = construct_square(l_img, x, y, sqsize)

    max_disparity = x - (0 + sqsize) # disaprity can be all the way up to the edge

    min_matching_cost = -1
    best_disparity = -1
    for i in range(0, max_disparity):
        r_square = construct_square(r_img, x - i, y, sqsize)

        matching_cost = compare_squares(base_l_square, r_square)
        if (matching_cost < min_matching_cost) or (i == 0):
            min_matching_cost = matching_cost
            best_disparity = i

    return best_disparity
```

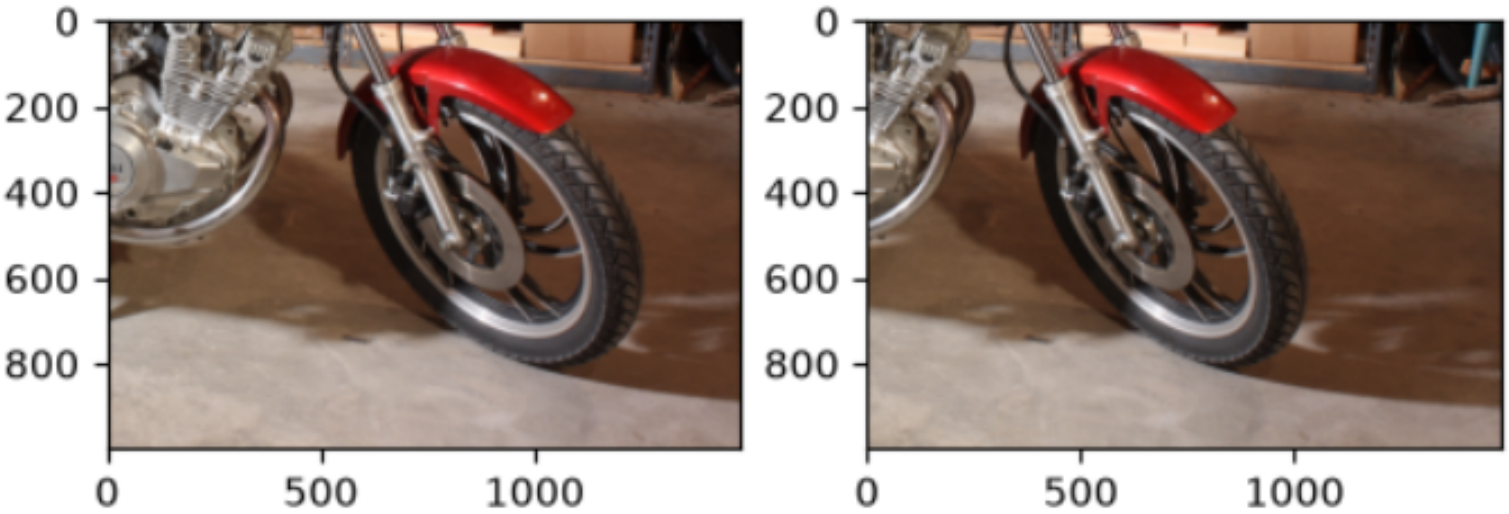
Simplest Algorithm in Python (2/2)

```python
def get_full_disparity_map(l_img, r_img, sqsize):
    h, w = l_img.shape[:2]
    disparity_map = np.zeros((h, w))

    for y in range(sqsize, h-sqsize):
        for x in range(sqsize, w-sqsize):
            disparity_map[y, x] = find_best_disparity(l_img, r_img, x, y, sqsize)
        print("Progress: ", y/h*100)
    return disparity_map
```

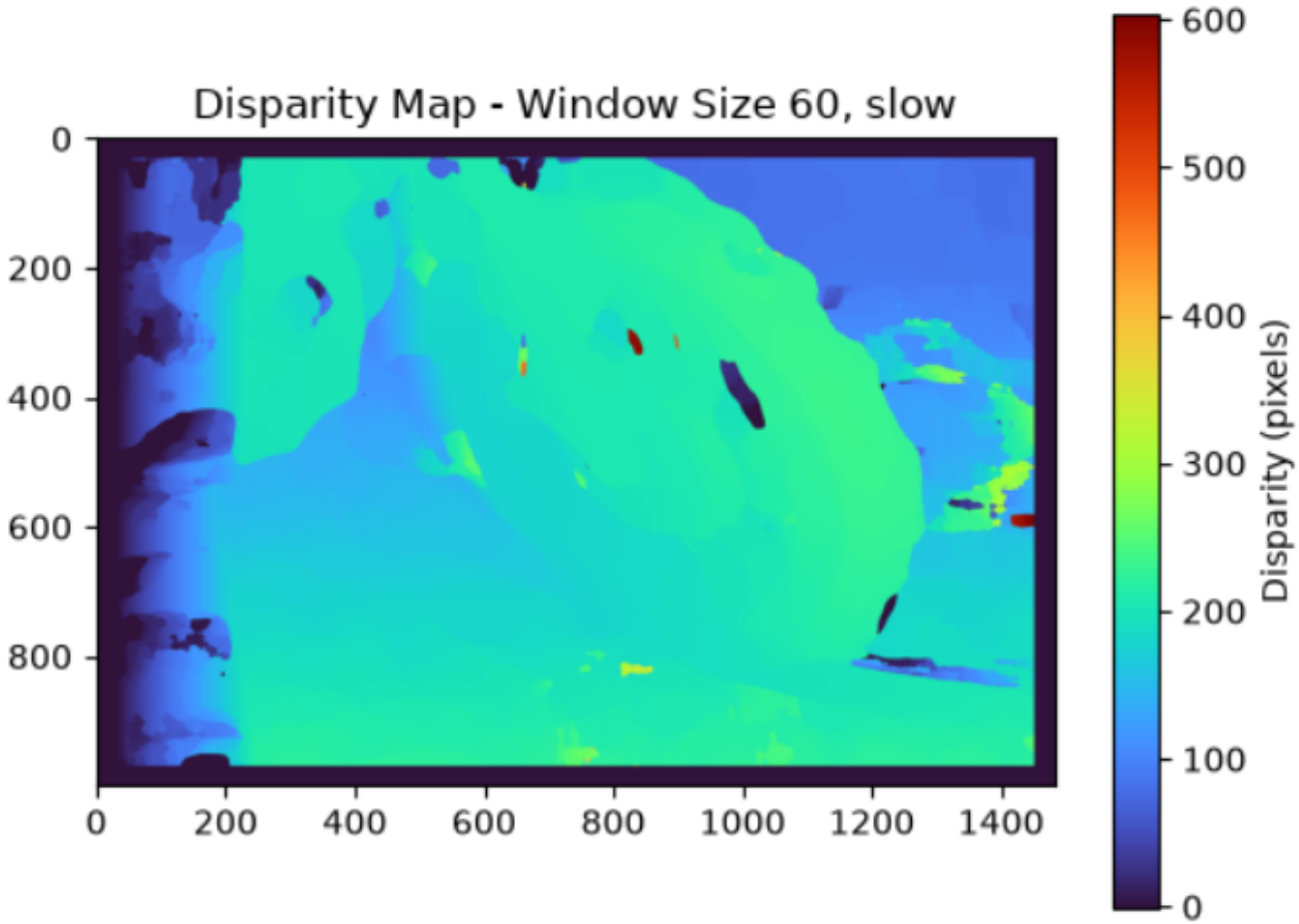
- Good, clear implementation
- Really, really slow

Testing on Some Images



- Images shifted
- 1000×1200 pixel images $\rightarrow$ took 1.5 hours

Testing on Some Images: Result



Speeding Things Up

- Vectorize cost computation between pixels all at once

```python
from scipy.ndimage import uniform_filter

def get_full_disparity_map_fast(l_img, r_img, sqsize, max_disparity):
    window_size = sqsize * 2
    h, w = l_img.shape[:2]
    cost_volume = np.zeros((h, w, max_disparity))

    for d in range(max_disparity):
        if d == 0:
            shifted_r_img = r_img
        else:
            shifted_r_img = np.zeros_like(r_img)
            shifted_r_img[:, d:] = r_img[:, :-d]

        abs_diff = np.sum(np.abs(l_img - shifted_r_img), axis=2)
        cost_volume[:, :, d] = uniform_filter(abs_diff, size=window_size)

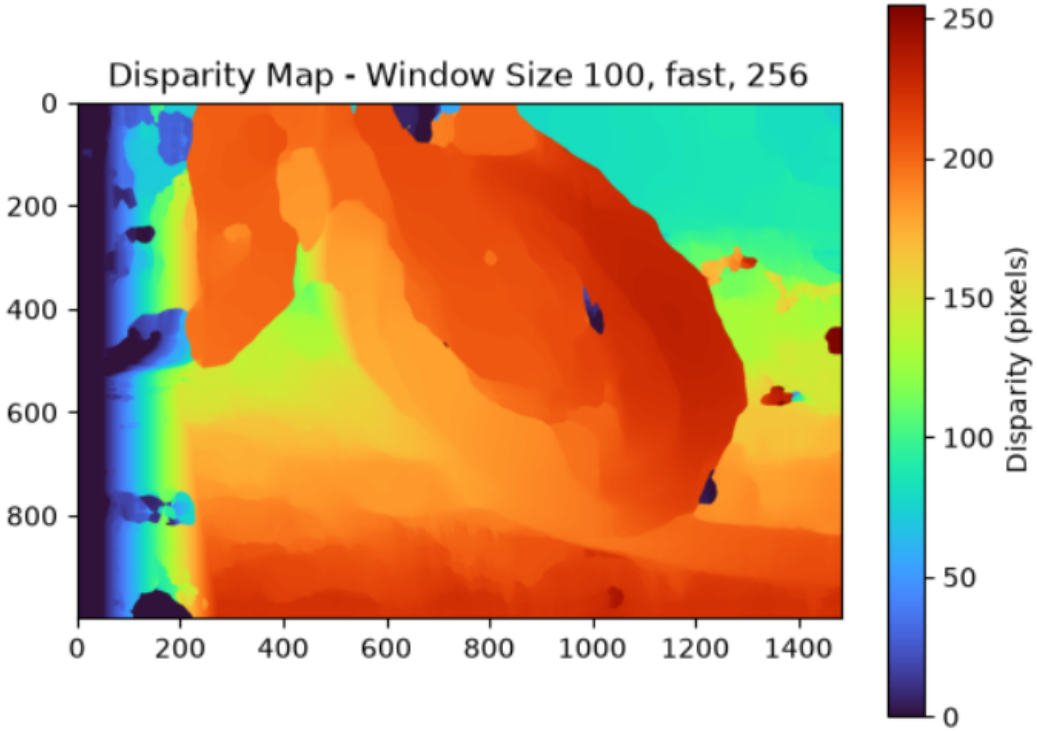
        print(f"\rProgress: {(d / max_disparity) * 100:.1f}%", end="", flush=True)


    disparity_map = np.argmin(cost_volume, axis=2)
    return disparity_map
```

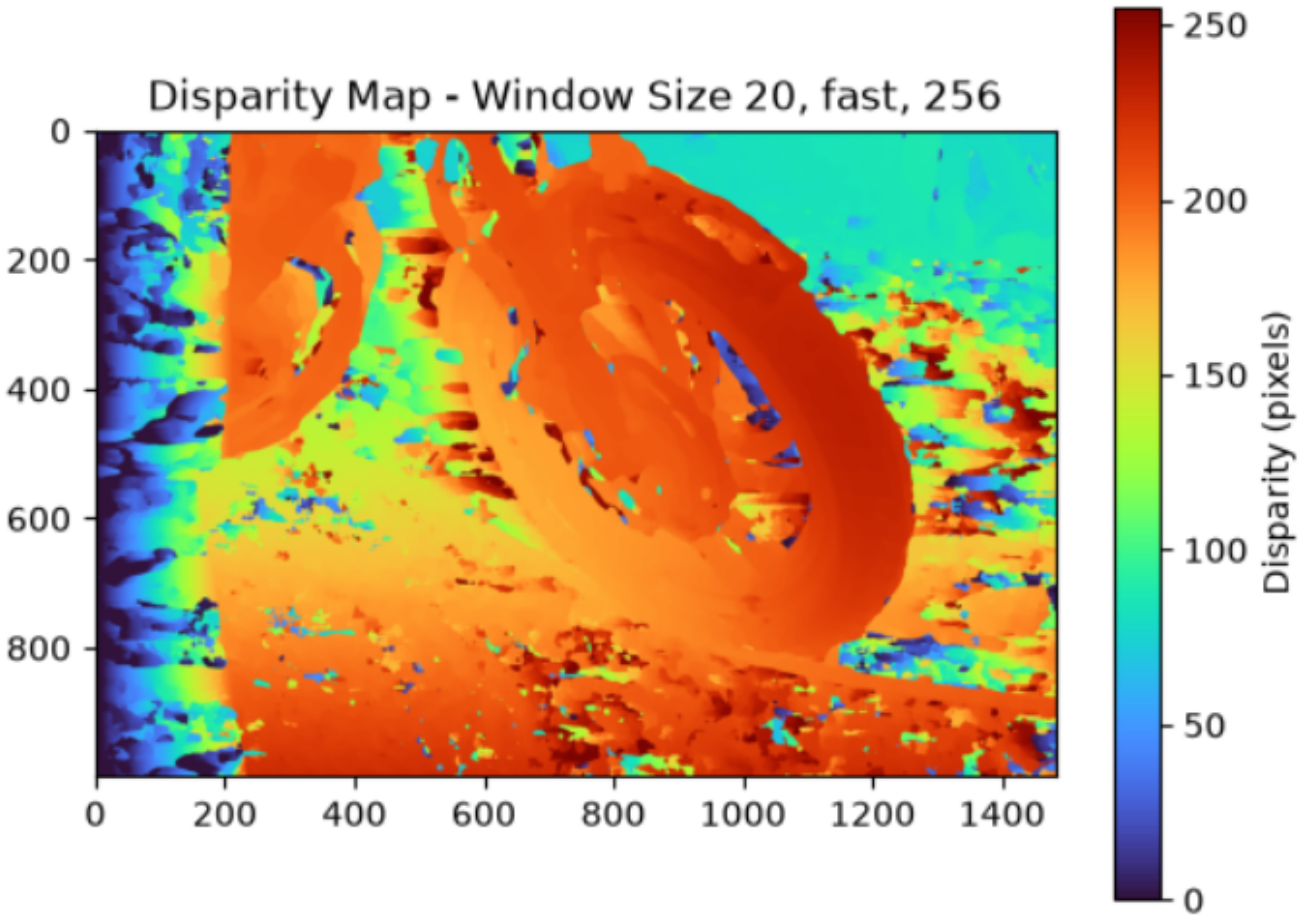
- Runs in about 1 minute

Varying Window Size: Window Size 100

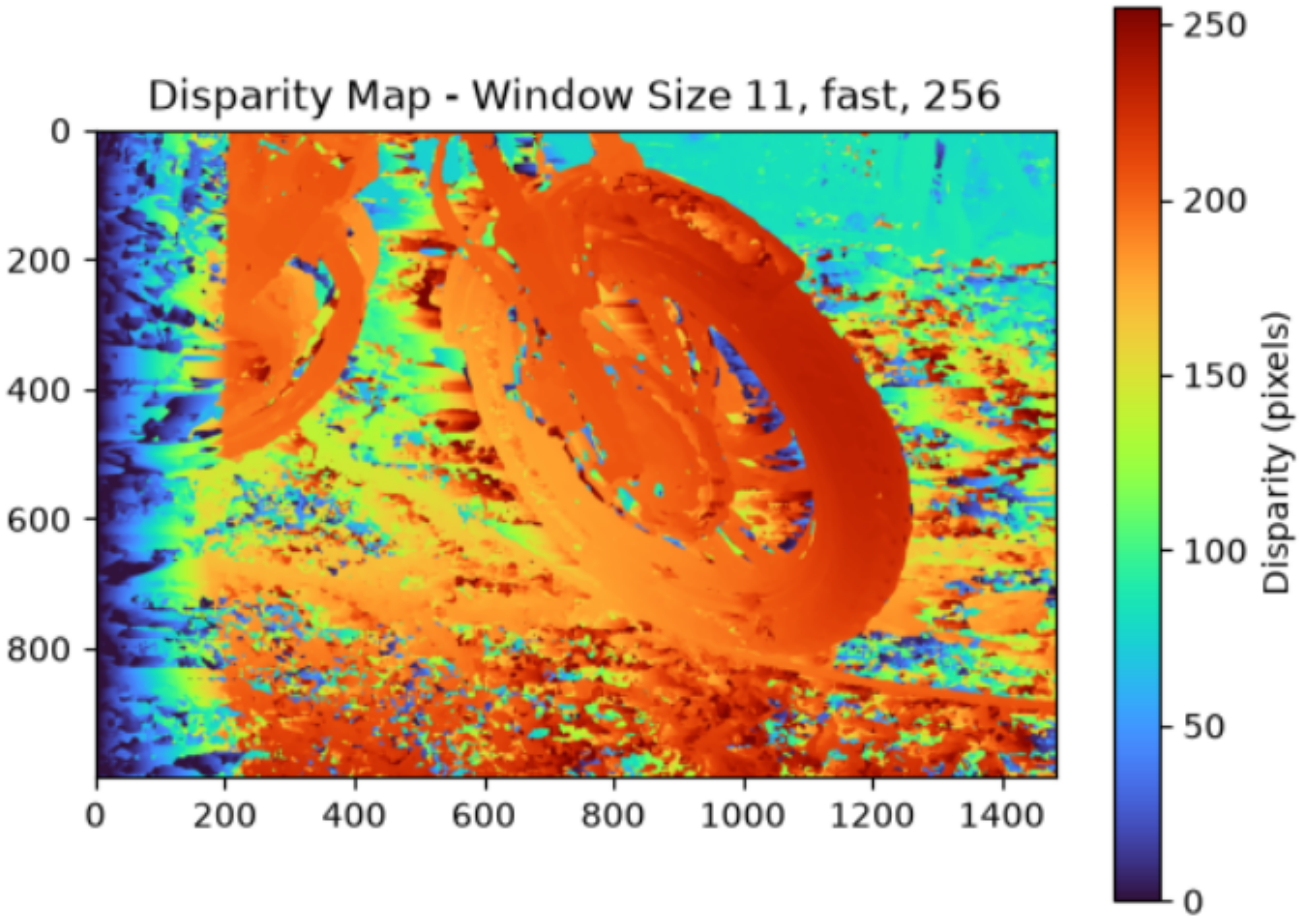
- Results vary greatly based on window size
- How big should comparison blocks be?



Varying Window Size: Window Size 20



Varying Window Size: Window Size 11



Beyond Block Matching

- Many improvements and better algorithms [3]:

Beyond Block Matching

- Many improvements and better algorithms [3]:
  - SGBM (Semi-Global Block Matching)
  - Global Optimization
  - Pre-Processing and Transforming Pixels
  - Refinement Methods

Beyond Block Matching

- Many improvements and better algorithms [3]:
  - SGBM (Semi-Global Block Matching)
  - Global Optimization
  - Pre-Processing and Transforming Pixels
  - Refinement Methods
- Good, can be fast
- Rely on human-designed procedure

Deep Learning Algorithms

- Train on large sets of images with known ground truth depth maps [4, 2]

Deep Learning Algorithms

- Train on large sets of images with known ground truth depth maps [4, 2]
- Pick up patterns and "real world" knowledge otherwise hard to detect

Deep Learning Algorithms

- Train on large sets of images with known ground truth depth maps [4, 2]
- Pick up patterns and "real world" knowledge otherwise hard to detect
- 2D algorithms: faster, don't compare disparity levels between pixels [2]

Deep Learning Algorithms

- Train on large sets of images with known ground truth depth maps [4, 2]
- Pick up patterns and "real world" knowledge otherwise hard to detect
- 2D algorithms: faster, don't compare disparity levels between pixels [2]
- 3D algorithms: slower, more depth awareness [2]

Deep Learning Algorithms

- Train on large sets of images with known ground truth depth maps [4, 2]
- Pick up patterns and "real world" knowledge otherwise hard to detect
- 2D algorithms: faster, don't compare disparity levels between pixels [2]
- 3D algorithms: slower, more depth awareness [2]
- Training data strongly influences performance:
  - Different environment (e.g., indoors vs. outdoors) $\rightarrow$ worse performance [2]

Project Goals

- Keep developing understanding of algorithms and tradeoffs:

Project Goals

- Keep developing understanding of algorithms and tradeoffs:
  - Preferred environment, speed, accuracy, hardware requirements

Project Goals

- Keep developing understanding of algorithms and tradeoffs:
  - Preferred environment, speed, accuracy, hardware requirements
- Try to train a model improving on one tradeoff:

Project Goals

- Keep developing understanding of algorithms and tradeoffs:
  - Preferred environment, speed, accuracy, hardware requirements
- Try to train a model improving on one tradeoff:
  - Idea: Combine existing training datasets
  - Idea: Add pre-processing step (object detection highlights?)

References


Google, *Gemini (large language model) — generated stereo vision diagrams*, `https://gemini.google.com`, 2026, AI-generated TikZ vector illustrations.


Mohd Saad Hamid, NurulFajar Abd Manap, Rostam Affendi Hamzah, and Ahmad Fauzan Kadmin, *Stereo matching algorithm based on deep learning: A survey*, Journal of King Saud University - Computer and Information Sciences **34** (2022), no. 5, 1663–1673.


Daniel Scharstein and Richard Szeliski, *A taxonomy and evaluation of dense two-frame stereo correspondence algorithms*, International Journal of Computer Vision **47** (2002), no. 1-3, 7–42.


Jure Žbontar and Yann LeCun, *Stereo matching by training a convolutional neural network to compare image patches*, Journal of Machine Learning Research **17** (2016), no. 65, 1–32.